

Trabajo Práctico

Visualización de algoritmos de
ordenamiento

Introducción a la programación

Integrantes:

Godoy Florencia

García Tomas

Laza Florencia

Introducción

La idea principal de este trabajo práctico fue implementar en Python varios algoritmos de ordenamiento. Lo interesante es que los algoritmos no se ejecutan “de golpe”, sino paso a paso, mostrando cada comparación y cada intercambio. Esta dinámica se visualiza en una página web construida con HTML y JavaScript, donde cada barra del gráfico representa un elemento de la lista. A medida que el algoritmo avanza, las barras se van moviendo y comparando en tiempo real, lo que facilita entender cómo funciona cada método de ordenamiento.

Para que esta visualización fuera posible, cada algoritmo tuvo que programarse de forma que pudiera pausarse y avanzar micro-paso por micro-paso. Los algoritmos obligatorios que implementamos fueron: Bubble Sort, Selection Sort e Insertion Sort. Además, como opcionales, agregamos Quick Sort y Heap Sort, lo que permitió trabajar con lógicas más complejas como la recursividad (simulada en forma iterativa) y la estructura de un heap binario.

Consigna del Trabajo Práctico (TP)

TP – Visualización de algoritmos de ordenamiento

Objetivo: Implementar al menos 3 algoritmos distintos de ordenamiento cumpliendo el contrato `init(vals)` + `step()` que usa el visualizador.

A implementar:

- Bubble Sort
- Selection Sort
- Insertion Sort.

Entregable:

- (Obligatorio) Carpeta `/algorithms/` con al menos 3 archivos funcionando.
- (Obligatorio) Informe detallado documentando los algoritmos implementados junto con las decisiones tomadas, implementaciones aplicadas y dificultades encontradas.
- (Obligatorio) Un `README.md` breve con: integrantes, algoritmos implementados y una nota corta sobre decisiones de implementación.

El informe debe incluir:

- Una introducción general no técnica.
- El código de los algoritmos implementados.
- Una breve explicación de cada algoritmo con dificultades y decisiones justificadas.
- No incluir explicaciones de funcionalidades de Python u otros frameworks.
- La conclusión debe incluir lo aprendido y los problemas

DESARROLLO

A continuación presentamos cada algoritmo, acompañado por su explicación y su fragmento de código en imagen.

Bubble Sort

Bubble Sort compara elementos adyacentes y los “empuja” hasta su posición final. El desafío no fue entenderlo, sino partir todo en micro-pasos: cada comparación, cada intercambio y cada avance tenía que mostrarse individualmente. También hubo que controlar los punteros para evitar que se pasaran del límite.

```
def step():
    global items, n, i, j

    if i >= n - 1:
        return {"done": True}

    if j >= n - 1 - i:
        i += 1
        j = 0
        return {"a": 0, "b": 0, "swap": False, "done": False}

    a = j
    b = j + 1
    swap = False

    if items[a] > items[b]:
        items[a], items[b] = items[b], items[a]
        swap = True

    j += 1
    return {"a": a, "b": b, "swap": swap, "done": False}
```

Insertion Sort

Insertion Sort va construyendo una parte ordenada y “desliza” cada nuevo elemento hacia su posición correcta.

Lo difícil fue simular ese deslizamiento sin usar while, sin hacer varios swaps juntos y respetando el modelo paso a paso.

```
def step():
    global items, n, i, j

    if i >= n:
        return {"done": True}

    if j is None:
        j = i
        return {"a": j - 1, "b": j, "swap": False, "done": False}

    if j > 0 and items[j - 1] > items[j]:
        items[j - 1], items[j] = items[j], items[j - 1]
        j -= 1
        return {"a": j, "b": j + 1, "swap": True, "done": False}

    i += 1
    j = None
    return {"a": 0, "b": 0, "swap": False, "done": False}
```

Selection Sort

Selection Sort no mueve elementos constantemente, primero recorre la parte no ordenada buscando el mínimo, y recién al final hace el intercambio.

La dificultad principal fue dividir este algoritmo en dos fases:

1. *búsqueda del mínimo*

2. *swap final*

```
def step():
    global items, n, i, j, min_idx, fase

    if i >= n - 1:
        return {"done": True}

    if fase == "buscar":
        if j >= n:
            fase = "swap"
            return {"a": i, "b": min_idx, "swap": False, "done": False}

        if items[j] < items[min_idx]:
            min_idx = j
        j += 1
        return {"a": min_idx, "b": j-1, "swap": False, "done": False}

    if fase == "swap":
        swap_hecho = False
        if min_idx != i:
            items[i], items[min_idx] = items[min_idx], items[i]
            swap_hecho = True

        a = i
        b = min_idx
        i += 1
        min_idx = i
        j = i + 1
        fase = "buscar"
        return {"a": a, "b": b, "swap": swap_hecho, "done": False}
```

Si no lo separábamos así, el visualizador no podía mostrar bien el proceso.

Shell Sort

Una versión mejorada de Insertion Sort que trabaja con saltos ("gaps").

El problema principal fue manejar correctamente el cambio de gap y reiniciar punteros sin romper el flujo del algoritmo.

```

def step():
    global items, n, gap, i, j

    if gap == 0:
        return {"done": True}

    if j is None:
        if i >= n:
            gap //= 2
            i = gap
            return {"a": 0, "b": 0, "swap": False, "done": False}

        j = i
        return {"a": j - gap, "b": j, "swap": False, "done": False}

    if j - gap >= 0 and items[j] < items[j - gap]:
        items[j], items[j - gap] = items[j - gap], items[j]
        j -= gap
        return {"a": j + gap, "b": j, "swap": True, "done": False}

    i += 1
    j = None
    return {"a": 0, "b": 0, "swap": False, "done": False}

```

Heap Sort

Probablemente el algoritmo más difícil del trabajo.

Tuvimos que reemplazar la recursión por una estructura iterativa y dividir la construcción del heap en pasos visibles. Cada “heapify” debía fraccionarse en pequeñas acciones para que el visualizador lo animara correctamente.

```

#fase build
if fase == "build":
    if j is None:
        j = i

    left = 2*j + 1
    right = 2*j + 2

    if left >= n:
        i -= 1
        j = None
        return {"a": i+1, "b": left, "swap": False, "done": False}

    mayor = left
    if right < n and items[right] > items[left]:
        mayor = right

    if items[mayor] > items[j]:
        items[j], items[mayor] = items[mayor], items[j]
        j = mayor
    return {"a": mayor, "b": j, "swap": True, "done": False}

i -= 1
j = None
return {"a": i+1, "b": mayor, "swap": False, "done": False}

```

Quick Sort

Quick Sort recursivo no podía implementarse directo, así que usamos una pila manual para simular la recursión.

La parte más complicada fue la partición, que normalmente ocurre de una vez, pero acá debía dividirse en micro-comparaciones, micro-swaps y cambios de punteros.

```

if fase == 1:
    if j < alto:
        out = {"a": j, "b": pivote_idx, "swap": False}

        if items[j] < pivot_v:
            i += 1
            if i != j:
                rep = swap(i, j)
                out["a"] = rep["a"]
                out["b"] = rep["b"]
                out["swap"] = True

        j += 1
        return out ➡

    else:
        fase = 2

if fase == 2:
    final_pos = i + 1
    did_swap = False

    if final_pos != alto:
        rep = swap(final_pos, alto)
        did_swap = True
        a = rep["a"]
        b = rep["b"]
    else:
        a = final_pos
        b = alto

    if bajo < final_pos - 1:
        pila.append((bajo, final_pos - 1))

    if final_pos + 1 < alto:
        pila.append((final_pos + 1, alto))

    fase = 0
    return {"a": a, "b": b, "swap": did_swap} ➡

```

Merge Sort

También intentamos implementar Merge Sort, pero finalmente tuvimos que descartarlo.

El problema principal fue que este algoritmo trabaja con sublistas auxiliares y bloques completos de elementos, mientras que el visualizador exige que cada `step()` represente una sola acción muy pequeña (una comparación o un swap).

Al dividir el algoritmo en micro-pasos, quedaban estados intermedios que no se podían mostrar correctamente o rompían la lógica del merge. Probamos distintas formas de simular la mezcla, pero ninguna se ajustaba al contrato `init()` + `step()`.

Por eso, decidimos no incluirlo y enfocarnos en que los demás algoritmos funcionaran correctamente.

Para entender mejor cómo funcionaba cada algoritmo, también buscamos recursos más visuales.

Vimos videos de gente que explicaba los algoritmos usando cartas de Yu-Gi-Oh y comparaciones con combates de Pokémon.

Aunque parezca algo poco técnico, a nosotros nos ayudó un montón porque cada carta representaba un valor, y las interacciones entre monstruos funcionaban como metáforas de los swaps, los duelos entre valores o el proceso de seleccionar un mínimo o moverse por una lista. Esto hizo que recordar el comportamiento de cada algoritmo fuera más intuitivo y menos abstracto.

Conclusión

En general, este TP nos hizo entender los algoritmos de una manera mucho más profunda de lo que esperábamos. Nos obligó a pensar paso por paso qué hace realmente cada uno, en vez de limitarnos a memorizar el pseudocódigo.

Lo que más aprendimos fue a manejar punteros, comparaciones y swaps con mucha precisión, porque cualquier error chiquito se veía directamente en la animación.

También nos dimos cuenta de que no todos los algoritmos se adaptan bien a este formato. Por ejemplo, Merge Sort lo intentamos varias veces, pero es un algoritmo que trabaja con bloques completos y no pudimos llevarlo a la estructura de un solo micro-paso, así que lo descartamos sin perder más tiempo.

Al final, más allá de los errores y las idas y vueltas, logramos que todos los algoritmos implementados funcionaran correctamente dentro del visualizador. Y lo más importante es que realmente entendimos cómo se mueve cada algoritmo “por dentro”, algo que de otra forma tal vez no hubiéramos visto tan claramente.